

Notițe DevOps

Linux · Docker · server — recapitulare

Documentul pornește de la notițele tale din `notite/` : aceleași lucruri, grupate pe teme și cu explicația de dedesubt. Trei feluri de conținut:

notiță ce ai scris tu, reorganizat · **corecție** ceva scris greșit, cu motivul · **completare** ceva ce *deja folosești*, dar n-aveai notat

1. Linux

Pachete

```
sudo apt update          # reîmprospătează lista de pachete
sudo apt install <nume>  # instalează
sudo apt remove <nume>   # dezinstatează
apt search <cuvant>       # caută prin pachete
```

`apt update` nu instalează nimic — doar află ce versiuni există acum. De aia se rulează întotdeauna prima: fără ea, `install` îți dă versiunea pe care o știa sistemul acum trei luni.

Useri și grupuri

```
sudo adduser <nume>      # cont nou
sudo groupadd <nume>     # grup nou
sudo usermod -aG <grup> <user> # adaugă userul într-un grup
sudo deluser <nume>      # șterge userul
cat /etc/passwd          # toți userii de pe mașină
```

DE CE „-aG” și nu „-G”

`-G` singur **înlocuiește** toate grupurile userului cu cel pe care îl dai. `-a` (append) doar adaugă. Dacă scrii `usermod -G docker ubuntu` pe EC2, userul iese din toate celelalte grupuri — inclusiv din `sudo` — și rămâi fără drepturi de administrator pe server.

Fișiere și permisiuni

```
ls -l          # cine poate citi / scrie / rula fiecare fișier
ls -la         # inclusiv ascunse (. = folderul curent, .. = părinte)
ls -ld <dir>    # despre DIRECTOR, nu despre ce e în el
chmod +x <fișier> # îl face rulabil ca program
chmod 740 <fișier>
sudo chown <user>:<grup> <fișier>
```

Cifrele se adună: `r` = 4, `w` = 2, `x` = 1. Deci `740` înseamnă 7 (4+2+1, tot) pentru tine, 4 (doar citire) pentru grup, 0 (nimic) pentru restul. Cele trei poziții sunt mereu în ordinea: **owner** · **grup** · **alții**.

Notița ta despre `chmod -R 777` e corectă, inclusiv avertismentul: dă tuturor drepturi complete. Pe laptop te deblochează, pe server e o gaură.

Pipe și redirectare

```
comanda1 | comanda2      # rezultatul primei intră direct în a doua
history | grep sudo      # caută „sudo” în istoricul comenzilor
comanda > fișier.txt      # SCRIE în fișier, ștergând ce era
comanda >> fișier.txt     # ADAUGĂ la finalul fișierului
cat <fișier>             # afișează tot conținutul
less <fișier>            # pagină cu pagină, „q” ca să ieși
```

Vim

```
vim <fisier>    # deschide sau creează  
i              # intri în modul scriere (Insert)  
Esc            # ieși din scriere, ca să poți da comenzi  
:wq           # salvează și închide  
:q!           # închide FĂRĂ să salveze
```

Script bash

```
#!/bin/bash          # prima linie: cu ce se citește scriptul  
chmod +x script.sh   # îi dai voie să ruleze  
./script.sh          # îl pornești  
$1, $2               # argumentele primite la pornire  
for f in *.txt; do echo $f; done # buclă peste fișiere
```

Rețea

```
ping <adresa>        # verifică dacă cealaltă mașină răspunde  
ip a                 # adresa IP a mașinii tale  
nslookup <domeniu>   # ce IP are un domeniu  
ps aux               # ce procese rulează acum
```

CORECȚIE — „ifconfig” nu mai există pe server

În notiță ai `ifconfig` pentru adresa IP. Comanda vine din pachetul `net-tools`, care **nu e instalat pe Ubuntu Server** — adică fix pe EC2-ul pe care lucrezi. Când o s-o tastezi acolo primești `ifconfig: command not found` și o să crezi că ai stricat ceva.

Comanda de azi e `ip a` (sau `ip addr`). `ifconfig` merge doar pe mașini unde cineva a instalat explicit `net-tools`.

SSH

```
ssh user@adresa_server  
ssh -i cheie.pem ubuntu@<IP_PUBLIC>  
ssh-keygen          # pereche de chei: privată + publică
```

Cheia privată nu pleacă niciodată de pe laptopul tău. Pe server urcă doar cea publică. De asta te poți conecta fără parolă: serverul verifică matematic că deții perechea, fără să afle vreodată secretul.

CORECȚIE — ce nu se scrie în notițe urcate pe GitHub

În `comenzi-server-aws.txt` ai linia `ssh -i aws-key.pem ubuntu@3.253.65.96` — IP public, user și numele cheii, toate trei împreună, într-un repo **public**. E jumătate din ce-i trebuie cuiva ca să încerce să intre.

În notițe se scrie `ubuntu@<IP_PUBLIC>`. Valorile reale stau într-un fișier local, negitat.

2. Docker — modelul de dedesubt

Imagine și container

Imaginea e un șablon înghețat: sistem minimal + dependențe + aplicația ta. **Containerul** e o instanță pornită din ea. Aceeași relație ca între clasă și obiect: dintr-o imagine pornești oricâte containere, și niciunul nu modifică imaginea.

CORECȚIE — cine alege porturile

În notiță ai scris:

```
ports:
  - "3310:3306"
3310 = portul calculator
3306 = port imagine (dockerhub) ----> NU IL ALEG EU
```

Prima linie e corectă. A doua, nu.

Numărul din dreapta nu vine „de la Docker Hub”. E portul pe care ascultă **procesul dinăuntru containerului**. La `mysql:8.4` se întâmplă să fie 3306 pentru că așa e pornit `mysqld` în imaginea aia — acolo, într-adevăr, nu-l alegi tu. Dar la aplicațiile tale îl alegi. Și chiar o faci — uite din `compose`-ul tău:

```
sebi-online-shop-app:
  ports:
    - "8093:8089"
  environment:
    SERVER_PORT: "8089"
```

8089 apare de două ori, și nu din întâmplare: îi spui lui Spring pe ce port să asculte, apoi scoți exact acel port afară, pe 8093.

Regula întreagă:

- **stânga** = port pe mașina ta; îl alegi liber, doar să nu fie ocupat;
- **dreapta** = portul pe care ascultă procesul din container;
- la imagini străine (`mysql`, `redis`, `nginx`) e fixat de imagine — îl afli din documentație; la aplicația ta îl decizi tu, prin `SERVER_PORT`.

Verificare: schimbă `SERVER_PORT` în 9000 și lasă `"8093:8089"`. Aplicația nu mai răspunde — nu pentru că e stricată, ci pentru că scoți afară un port pe care nu mai ascultă nimeni.

COMPLETARE — volume (le folosești, n-ai notă)

În `compose`-ul tău ai:

```
volumes:
  - mysql_all_data:/var/lib/mysql
```

Containerul e de unică folosință: când îl ștergi, dispăre tot ce a scris în el. Volumul e un sertar separat, care supraviețuiește containerului. `/var/lib/mysql` e locul unde MySQL își ține bazele — deci volumul ăsta e baza ta de date.

```
docker volume ls
docker compose down      # șterge containerele, PĂSTREAZĂ volumul
docker compose down -v   # șterge ȘI volumul = ai pierdut baza
```

Acel `-v` e toată diferența dintre „repornesc” și „am pierdut datele”.

COMPLETARE — de ce merge „mysql:3306” și nu „localhost”

În `compose`-ul tău scrie:

```
DB_URL: jdbc:mysql://mysql:3306/online_shop_db
```

`mysql` nu e un hostname pe care l-ai definit undeva — e **numele serviciului** din `compose`. `Compose` creează o rețea privată pentru fișierul tău și înregistrează fiecare serviciu în DNS-ul ei, cu numele lui.

`localhost` dintr-un container înseamnă *containerul însuși*. Dacă aplicația ar cere `localhost:3306`, s-ar căuta pe ea, n-ar găsi MySQL și ai primi `Connection refused`.

Consecința pe care merită s-o reții: **porturile publicate nu-i trebuie aplicației**. Ea vorbește pe rețeaua internă, direct pe 3306. Linia `"3307:3306"` de la MySQL e doar ca să te uiți TU din DataGrip. Pe server, scoate-o.

COMPLETARE — cum intri într-un container

```
docker logs -f <nume>           # ce scrie aplicația, în timp real
docker exec -it <nume> bash      # deschizi un shell ÎNĂUNTRU
docker inspect <nume>           # configurația reală a containerului

docker exec -it mysql-shared \
mysql -uroot -proot -e "SHOW DATABASES;"
```

Când containerul pornește dar aplicația nu merge, `logs` îți spune ce crede ea că s-a întâmplat, iar `exec` te lasă să verifici tu: chiar există baza? chiar e setată variabila? Fără astea două rămâi la ghicit.

3. Fluxul tău în 5 pași

Ordinea din notița ta e bună și e gândită ca proces, nu ca listă de comenzi — fiecare pas verifică ceva înainte să adaugi complexitate:

```
Pas 1: pornesc compose-ul cu serverul MySQL
Pas 2: verific în DataGrip, și aplicația rulând din IntelliJ
Pas 3: creez jar-ul și îl rulez, verific
Pas 4: Dockerfile: build and push
Pas 5: completez docker compose cu aplicația
```

CORECȚIE — două lucruri la pasul 3

a) Jar-ul se face cu `mvn package`, nu `mvn install`. `package` e pasul care produce `target/*.jar`. `install` face același lucru și în plus copiază artefactul în `~/.m2` — util doar când altă aplicație de-a ta depinde de el ca bibliotecă.

```
mvn package           # produce target/*.jar
java -jar target/<nume>.jar # verifici că pornește singur
```

b) Cu Dockerfile-ul multi-stage pe care îl ai deja, nu mai trebuie să faci jar-ul local pentru build. Primul stage îl construiește în container:

```
FROM eclipse-temurin:21-jdk AS build
RUN ./mvnw -q -DskipTests package
```

Deci pasul 3 rămâne în flux, dar își schimbă rolul: nu mai e pas de *build*, e pas de **verificare**. Și e valoros exact așa — dacă `java -jar` merge local dar containerul nu merge, știi din prima că problema e în Dockerfile sau în compose, nu în cod. Asta îți taie jumătate din locurile în care ai căuta.

4. Comenzi Docker

```
docker ps           # containere pornite
docker ps -a        # inclusiv cele oprite
docker images       # imaginile de pe mașină
docker --version
docker compose version
docker run --rm hello-world # test că Docker chiar funcționează
```

Build și push

```
docker build -t sebi123q/online-school-cloud-api:0.0.1 .
docker push sebi123q/online-school-cloud-api:0.0.1
```

Ai notat corect că `-t` e doar la `build`: la `push` dai numele deja tag-uit. E un detaliu pe care mulți îl greșesc luni de zile. Punctul de la finalul lui `build` nu e decor — e **contextul**: folderul pe care Docker îl împachetează și îl trimite ca să construiască imaginea. De aia contează ce ai în folderul ăla (revenim la asta cu `.dockerignore`).

Compose

```
docker compose up -d          # pornește tot, în fundal
docker compose down          # oprește și șterge containerele
docker compose logs -f       # loguri live
docker compose restart
docker compose -f <fișier>.yaml up -d  # alt fișier decât cel implicit
docker compose -f <fișier>.yaml config  # VERIFICĂ fișierul, fără pornire
```

DOUĂ PLASE DIFERITE, PENTRU DOUĂ GREȘELI DIFERITE

`config` îți spune dacă fișierul e valid ca structură — indentare greșită, cheie inexistentă, serviciu pus unde nu trebuie. Rulează-l **înainte de fiecare commit**: durează o secundă și nu pornește nimic.

Dar `config` verifică *structura*, nu *valorile*. Un URL de bază de date scris greșit trece de `config` fără nicio plângere și cade abia la pornire. Pe ăla îl prinzi doar cu `logs`, după `up`.

Deci: `config` înainte de commit, `logs` după `up`. Amândouă, de fiecare dată.

5. Server AWS (EC2)

Pașii tăi de pe server, curățați și puși în ordine:

```
ssh -i aws-key.pem ubuntu@<IP_PUBLIC>

sudo apt update
sudo apt install -y docker.io
sudo apt install -y docker-compose-v2
sudo systemctl enable docker  # să pornească singur după reboot
sudo systemctl start docker   # pornește-l acum
sudo usermod -aG docker $USER  # ca să rulezi docker fără sudo
exit                          # ieși, ca să se aplice grupul
docker ps                     # după reconectare
```

„enable” ȘI „start” — nu e redundanță

`start` pornește serviciul *acum*. `enable` îl înregistrează ca să pornească *și după reboot*. Sunt lucruri diferite și îți trebuie amândouă: fără `enable`, la prima repornire a serverului toate containerele tale rămân jos și nu înțelegi de ce. Le-ai făcut pe amândouă — bine.

`usermod -aG docker` se aplică abia la următoarea conectare. De asta urmează `exit`: nu e o ieșire oarecare, e pasul care activează grupul.

CORECȚIE — ai două metode de instalare, în două fișiere

În `LinuxHandout.txt`:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

În `comenzi-server-aws.txt`:

```
sudo apt install -y docker.io docker-compose-v2
```

Pe EC2 ai folosit-o pe a doua. Amândouă funcționează, dar dau **versiuni diferite** de Docker: scriptul oficial aduce ultima versiune de la Docker, pachetul `docker.io` aduce versiunea din Ubuntu, mai veche.

Scrie în notiță care ai folosit. Altfel, peste o lună, pe alt server, o să vezi altă versiune și n-o să știi dacă ai greșit ceva sau e doar altă metodă.

6. curl — pe proiectele tale

Notița ta de `curl` lovește `localhost:8085/api/departments`. Nu ai nici portul 8085, nici resursa `departments` în vreun proiect — e o fișă copiată, pe care ar trebui s-o traduci de fiecare dată. Aici e aceeași fișă, dar pe shop-ul tău (host

8093 , din compose-ul tău):

```
curl http://localhost:8093/api/products
curl -i http://localhost:8093/api/products      # și status + headere
curl -s -o /dev/null -w "%{http_code}\n" \
  http://localhost:8093/api/products           # doar codul: 200, 404...
```

COMPLETARE — token-ul într-o variabilă

Toate API-urile tale sunt securizate cu JWT, iar acum lipești token-ul de mână la fiecare cerere (Bearer TOKENUL_TAU). Se poate o singură dată:

```
TOKEN=$(curl -s -X POST \
  http://localhost:8093/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"...","password":"..."}' \
  | jq -r '.accessToken')
```

```
curl http://localhost:8093/api/orders \
  -H "Authorization: Bearer $TOKEN"
```

jq taie din răspunsul JSON doar câmpul accessToken (ăsta e numele exact din AuthResponse -ul tău) și îl pune în variabilă. Din momentul ăla, orice cerere folosește \$TOKEN . Instalare: brew install jq .

```
curl -X POST http://localhost:8093/api/products \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Laptop","price":3500}'

curl -X DELETE http://localhost:8093/api/products/1 \
  -H "Authorization: Bearer $TOKEN"
```

7. Hardening SSH — din listă în comenzi

În Hardning.txt ai patru rânduri, și toate patru sunt corecte: autentificare cu cheie, root nu se poate conecta, parola dezactivată, portul 22 doar de la IP-ul tău. Ce lipsește e partea care se tastează.

```
# 1. urci cheia publică pe server
ssh-copy-id -i ~/.ssh/id_ed25519.pub ubuntu@<IP_PUBLIC>

# 2. editezi configul de SSH, pe server
sudo vim /etc/ssh/sshd_config
    PermitRootLogin no
    PasswordAuthentication no
    PubkeyAuthentication yes

# 3. aplici
sudo systemctl restart ssh
```

REGULA DE AUR ÎNAINTE DE „restart ssh”

Înainte să dai restart, deschide un **al doilea terminal** și verifică acolo că te poți conecta cu cheia. Dacă ai greșit ceva în sshd_config și închizi singura sesiune deschisă, nu mai ai cum să intri pe server — și îl reconstruiești de la zero. Sesiunea veche rămâne validă după restart; ea e plasa ta de siguranță.

Al patrulea punct — *portul 22 doar de la IP-ul tău* — nu se face din Linux, ci din **Security Group**-ul instanței, din consola AWS: la regula pentru portul 22, schimbi sursa din 0.0.0.0/0 în <IP-ul tău>/32 .

CORECȚIE — ce ai deschis acum fără să vrei

În compose-urile tale publici MySQL: `ports: "3307:3306"`, cu `MYSQL_ROOT_PASSWORD: root`. Pe laptop e doar neglijent. Pe server, orice port din `ports:` e vizibil din internet dacă Security Group-ul îl lasă — iar scanerile automate găsesc un MySQL cu root/root în câteva minute.

Aplicațiile tale **nu au nevoie** de portul publicat: vorbesc cu MySQL pe rețeaua internă. Pe server, șterge linia. Dacă vrei totuși acces din DataGrip, leagă-l doar local: `"127.0.0.1:3307:3306"`, și intri prin tunel SSH.

8. Ce urmează

Notițele tale se opresc exact acolo unde continuă cursul: ai serverul, ai Docker pe el, ai aplicațiile care rulează — dar fiecare cu portul ei scos afară. Pasul următor e **o singură poartă**: nginx în față, pe portul 80, care trimite mai departe către aplicații, iar toate porturile lor rămân legate doar local.

Atunci se închid, dintr-o mișcare, și cele patru puncte din `Hardning.txt`, și problema cu MySQL expus.